Self-Deploy — Multi-Factor Authentication & Conditional Access

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Self-Deploy: Multi-Factor Authentication & Conditional Access 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

What this delivers . . . 6

The standard deployment model . . . 6

Outcome . . . 6

Important boundaries . . . 6

1. Solution Overview — Self-Deploy: Multi-Factor Authentication & Conditional Access 6

What this is . . . 7

How the technology works (the mental model) . . . 7

The standard deployment model . . . 8

Scope — what this kit covers . . . 9

Out of scope (handled elsewhere) . . . 9

Benefits . . . 9

Limitations (be honest with buyers) . . . 9

Authentication and connections used . . . 10

2. Requirements Guide — Self-Deploy: Multi-Factor Authentication & Conditional Access 10

R1. Entra ID licensing for Conditional Access (P1; P2 for risk labs) . . . 10

R2. A learner cohort and least-privilege lab roles . . . 11

R3. Lab persona groups (the lab scope) . . . 11

R4. Break-glass accounts and an exclusion group . . . 11

R5. MFA methods understood; sign-in logs available . . . 12

R6. The PowerShell + KQL helper toolchain . . . 12

Checklist . . . 12

Global Requirements — Entra ID Solutions (category-level) 13

G1. Microsoft Entra ID tenant . . . 13

G2. Entra ID licensing (P1, P2 where noted) . . . 13

G3. Administrative roles . . . 13

G4. Break-glass (emergency access) accounts . . . 13

G5. Healthy on-premises Active Directory . . . 14

G6. An Entra Connect host server . . . 14

G7. UPN / domain hygiene . . . 14

G8. Network access to Microsoft cloud endpoints . . . 14

3. Implementation Guide — Self-Deploy: Multi-Factor Authentication & Conditional Access 14

Phase 0 — Assess (read-only, changes nothing) . . . 15
Phase 1 — Configure (persona groups + break-glass exclusion) . . . 16
Phase 2 — Pilot (the guided labs, all report-only) . . . 16
Phase 3 — Validate (capstone + knowledge-check) . . . 18
Phase 4 — Expand (take the capability to production) . . . 18
Rollback / safe-off . . . 19
5. Testing & Validation — Self-Deploy: Multi-Factor Authentication & Conditional Access 19
Gate 1 — CA is available and the MFA/CA picture is captured . . . 19
Gate 2 — Cohort, persona groups, and break-glass exclusion exist . . . 19
Gate 3 — The report-only lab policies exist and are report-only . . . 20
Gate 4 — Break-glass is excluded from every lab policy . . . 20
Gate 5 — What-If matches the persona design . . . 20
Gate 6 — Learners can read a sign-in and explain the CA decision . . . 20
Gate 7 — The knowledge-check passes . . . 21
Gate 8 — Report-only-only posture held; roles least-privilege . . . 21
6. Troubleshooting — Self-Deploy: Multi-Factor Authentication & Conditional Access 22
1. `Connect-MgGraph` fails or a script returns 403 . . . 22
2. The report-only lab policy can't be created (CA blade missing) . . . 22
3. A lab policy is not in report-only . . . 22
4. Break-glass is not excluded from a lab policy . . . 23
5. What-If shows a policy applying to the wrong persona (or not at all) . . . 23
6. A "failure" in the sign-in log is actually a report-only result . . . 23
7. MFA prompts constantly / a phishing-resistant policy rejects an admin . . . 23
8. Sign-in logs / the KQL lab return no rows . . . 24
9. Legacy-authentication confusion . . . 24
10. Dynamic persona-group membership didn't update . . . 24
7. Best Practices — Self-Deploy: Multi-Factor Authentication & Conditional Access 24
Report-only before enforce — always . . . 24
Protect break-glass, every policy, no exceptions . . . 25
Choose MFA methods by strength . . . 25
Design by persona, not by mega-policy . . . 25
Troubleshoot from the sign-in log, not from a guess . . . 25
Verify, don't assume . . . 26
Run it as a program, not a one-off class . . . 26
Where to go next . . . 26
8. Deliverables Summary — Self-Deploy: Multi-Factor Authentication & Conditional Access 26

Documentation 27
Lab helpers (5) 28
Named objects (verbatim — used across docs, scripts, and tests) 29
What this kit deliberately does NOT do 29
How it is delivered 30
Outcome 30
Appendix A — PowerShell Scripts 31

Executive Summary — Self-Deploy: Multi-Factor Authentication & Conditional Access

White-label · Entra ID Solutions

The problem

Multi-Factor Authentication and Conditional Access are the front line of identity security in Microsoft Entra — and they are also the single easiest place to lock the whole company out of its own tenant. A team that does not deeply understand MFA methods, how a Conditional Access policy is evaluated, and how to test a policy *before* it can block anyone tends to swing between two failures: too timid (gaps left open because nobody dares turn a policy on) or too reckless (an overbroad policy saved straight to enforcement that locks out real users, or worse, the admins). The missing thing is not another policy — it is a team that can **design, test, and troubleshoot** MFA and CA safely.

The solution

This is a **self-guided self-deploy** — concepts, **guided hands-on labs**, a **take-home runbook**, and a **knowledge-check** — that levels a team up on Microsoft Entra MFA and Conditional Access. It is deliberately a **skills** product and is **distinct from the CA implementation solutions** in this catalog (the Zero Trust Conditional Access Baseline and Conditional Access Integration *deploy and operate* a production baseline). This kit builds the **capability**: the team learns MFA methods (including phishing-resistant), CA design patterns and personas, and — most importantly — the **report-only + What-If** discipline that lets them practice on real Entra **with no lockouts**.

- **Teaches MFA methods properly** — from SMS (weak) through authenticator to phishing-resistant FIDO2/passkeys, plus registration and the authentication-methods policy.
- **Teaches CA by design, not by accident** — the anatomy of a policy, personas (admins/internals/guests/service accounts), and the common baseline patterns — as *understanding*, not a copy-paste.
- **Ships report-only, no-lockout labs** — every lab policy is built in **report-only**, validated with the **What-If** tool, with **break-glass** protected throughout — so learning is safe.
- **Teaches troubleshooting** — how to read a sign-in failure and trace exactly why a Conditional Access policy did or did not apply.

What you get

- A plain-language **Solution Overview** of MFA methods and the Conditional Access model — signals, conditions, grant/session controls, personas, report-only, What-If, and break-glass.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (on top of the shared category requirements).

• An **Implementation Guide** structured as guided labs — MFA methods & registration, CA anatomy, CA

design patterns & personas, building **report-only** policies, **What-If** testing, and sign-in/CA-evaluation troubleshooting — then a capstone and knowledge-check.

• **Five lab helpers** — a read-only MFA/CA discovery, a gated report-only lab-policy builder (preview

by default), a sign-in/CA-evaluation **KQL lab pack**, a read-only validation/knowledge-check runner, and a report-only-impact + cohort report.

• **Testing and Troubleshooting** guides so the team learns to validate and self-diagnose.

What this delivers

A team that can **design, test, and troubleshoot** MFA and Conditional Access on Microsoft Entra safely — choosing the right MFA methods, building CA policies by persona, proving every policy in report-only and What-If before it can enforce, protecting break-glass, and reading a sign-in log to explain a CA decision — plus a take-home runbook and a passed knowledge-check.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand — reframed for enablement: assess readiness, configure the cohort and lab persona groups, run the guided labs (pilot) in report-only, validate skills with a capstone and knowledge-check, then expand into the real CA baseline solutions.

Outcome

Confident MFA and Conditional Access policies designed, tested, and troubleshot by a team that understands them — no lockouts, no gaps — and a clear on-ramp to the production CA implementation solutions.

Important boundaries

This is a **skills / enablement** product — it teaches a team to design, test, and troubleshoot MFA and CA. It is **adjacent to, and deliberately does not duplicate**, the catalog's CA **implementation** solutions: the **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** *design and roll out a production, enforced baseline*; this kit builds the *team capability* and practices in **report-only** only. It cross-references the **Passwordless & Passkey (ENTPASSKEY)**, **PIM (ENTPIM)**, **Identity Governance (ENTIGA)**, and **Hybrid Identity** solutions rather than repeating them. Every lab policy is **report-only** and can never enforce; break-glass is protected throughout. Where a feature needs a specific license (Entra ID P1 for CA, **P2** for risk-based conditions and PIM) or is in preview, this kit flags it — *verify current availability and licensing in your tenant*, which can differ by cloud (Commercial vs GCC / GCC High).

1. Solution Overview — Self-Deploy: Multi-Factor Authentication & Conditional Access

What this is

A complete, self-guided **self-deploy** that levels a team up on **Multi-Factor Authentication and Conditional Access** in Microsoft Entra — MFA methods (including phishing-resistant), the Conditional Access model, CA design patterns and personas, **report-only + What-If** testing, break-glass, and troubleshooting sign-in and CA-evaluation failures. It is delivered as concepts, **guided hands-on labs** (all in report-only), a **take-home runbook**, and a **knowledge-check**.

It is a **skills** product — the outcome is a team that can design, test, and troubleshoot MFA/CA safely. It is deliberately **distinct from** the catalog's CA **implementation** solutions — the **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** *deploy and operate* a production, enforced baseline. This kit builds the *capability* and practices in **report-only**; those products build the production system. The two are complementary and are cross-referenced, not duplicated.

How the technology works (the mental model)

MFA — methods and strength

- **MFA adds a second factor** beyond the password. The methods are **not equal in strength**:
- **SMS / voice** — better than nothing, but **phishable** and SIM-swap-prone.
- **Microsoft Authenticator (push + number matching)** — strong and common; number matching resists

MFA-fatigue attacks.

- **Phishing-resistant methods — FIDO2 security keys, passkeys, Windows Hello for Business,

certificate-based auth** — the strongest tier; they are bound to the device/origin and cannot be phished. *Passkey support and some methods may be in preview or licensing-gated — verify.*

- **The authentication-methods policy** (Entra -> Protection -> Authentication methods) controls

which methods are enabled and for whom, and drives **registration** (including registration campaigns that nudge users to set up a stronger method).

- **Authentication strength** is a named requirement (e.g. "Phishing-resistant MFA") you can require

in a Conditional Access grant control — the bridge between MFA methods and CA.

Conditional Access — the if-then engine

A Conditional Access policy is an **if-then** statement evaluated on every sign-in **after** authentication and **before** a token is issued:

- **The "if" — assignments and conditions:** which **users/groups** (and **exclusions**), which

target resources (cloud apps / user actions), and conditions such as **device platform**, **location**, **client app**, **sign-in risk**, and **user risk**. *Risk conditions require Entra ID **P2** (Identity Protection) — verify.*

- **The "then" — controls:**
- **Grant controls** — allow only if a requirement is met: **require MFA**, **require a specific

authentication strength **(e.g. phishing-resistant),** require a compliant device**,** require a hybrid-joined device**,** require app-protection policy**, or** block**. Combine with AND/OR.

• **Session controls** — shape the granted session: **sign-in frequency**, **persistent browser**, **app-enforced restrictions**, and CA app control.

Because a policy can target "all users" and "all cloud apps," it is powerful enough to lock everyone — including admins — out in one save. The whole method below exists to prevent that.

The two safety features every learner must master

• **Report-only mode.** A policy set to **Report-only** is fully evaluated and logged on every sign-in ("would have granted / would have blocked / would have required MFA") **without enforcing anything**. You read the impact in **Conditional Access -> Insights and reporting** and in the sign-in logs. *Every lab policy in this kit is report-only.*

• **What-If.** A simulator (**Conditional Access -> What If**) where you pick a user, app, device state, and conditions and see exactly which policies would apply and which controls would fire — with no real sign-in. It is how you sanity-check a policy against each persona before it is ever enforced.

Break-glass — the anti-lockout discipline

Two cloud-only emergency-access (break-glass) accounts are **excluded from every CA policy** (global requirement **G4**) so a bad policy can never lock all admins out. The kit teaches this habit on every lab policy, even though the lab policies are report-only.

Design by persona

The clean way to think about CA is by **persona**, not one mega-policy: **admins** (highest value — phishing-resistant MFA, tight session), **internal staff** (MFA + compliant/hybrid device), **external/guest** (MFA + app-enforced restrictions), and **service accounts / workload identities** (non-interactive — scoped out of interactive policies, governed separately). Personas map to security groups, and policies target those groups.

The standard deployment model

Every solution in this program follows the same five stages; here they structure the kit:

1. **Assess.** Confirm the cohort and tenant are ready — CA is available (P1), MFA methods are understood, break-glass exists (run `EKMFACA-Discovery.ps1` for the read-only picture).
1. **Configure.** Create the learner cohort `Grp-MfaCa-Cohort`, the lab **persona groups**, and the break-glass exclusion group — a small, cloud-only lab scope.
1. **Pilot.** Run the **guided labs** (Labs 1–6) — MFA methods & registration, CA anatomy, design patterns & personas, building **report-only** policies, **What-If** testing, and sign-in/CA-evaluation troubleshooting.
1. **Validate.** Run the **capstone** and score the **knowledge-check** `Check-MfaCa-Pilot` (`EKMFACA-Validate.ps1`); confirm every lab policy is report-only and excludes break-glass.
1. **Expand.** Take the capability into the **production** CA solutions (ENTZTCA, Conditional Access Integration) to design and roll out a real, enforced baseline safely.

Scope — what this kit covers

• **MFA methods & registration** — the method tiers, phishing-resistant options, the authentication-methods policy, and registration.

• **Conditional Access anatomy** — assignments, conditions, grant and session controls, exclusions.

• **CA design patterns & personas** — admins, internals, guests, service accounts, and the common baseline patterns as *understanding*.

• **Report-only + What-If testing** — building and validating policies with **no lockouts**.

• **Break-glass** — the anti-lockout discipline on every policy.

• **Troubleshooting** — reading sign-in failures and CA-evaluation results to explain a decision.

• **A take-home runbook + knowledge-check** — so the skills persist and can be proven.

Out of scope (handled elsewhere)

• **Designing and enforcing a production CA baseline.** That is the **Zero Trust Conditional Access Baseline (ENTZTCA) **and** Conditional Access Integration **solutions. This kit practices in** report-only** and never enforces.

• **Rolling out phishing-resistant methods at scale.** Deploying FIDO2/passkeys across the org is the **Passwordless & Passkey (ENTPASSKEY)** solution — introduced here, delivered there.

• **Risk-based access, PIM, governance, hybrid.** Identity Protection risk policies (P2), just-in-time admin (ENTPIM), access reviews (ENTIGA), and AD sync (Hybrid Identity) are cross-referenced, not built.

• **The business/security determinations.** Which MFA methods are mandated, which users are "privileged," how strict guest access must be, and every production enforcement decision are the customer's security/operations calls; this kit provides the skill and the labs.

Benefits

• **No lockouts while learning** — every lab policy is report-only and validated with What-If; break-glass is protected throughout.

• **No gaps from timidity** — the team understands MFA/CA well enough to design and test confidently.

• **Troubleshooting skill** — the team can read a sign-in log and explain why a policy applied, cutting "MFA is broken" tickets.

• **A safe on-ramp to production** — the capability feeds directly into the enforced-baseline solutions.

Limitations (be honest with buyers)

• This teaches **capability** and practices in **report-only**; it is not a production baseline (that

is ENTZTCA / Conditional Access Integration).

• Conditional Access needs **Entra ID P1**; **risk-based conditions** and PIM need **P2**; some MFA methods (passkeys, CBA) or features may be in preview — *verify availability in your tenant*.

• Feature names, portal navigation, and licensing bundles evolve and can differ by cloud (Commercial vs GCC / GCC High) — *verify against your tenant*.

Authentication and connections used

• **Microsoft Entra admin center** (`https://entra.microsoft.com`) for the interactive labs — Authentication methods, Conditional Access policies, What-If, Insights and reporting, and Sign-in logs.

• **Microsoft Graph PowerShell SDK** for the read-only discovery/validation/reporting helpers and the gated, preview-by-default **report-only** lab-policy builder (each script's header names its least-privilege scopes).

• **KQL** over Entra **sign-in logs** (via a Log Analytics workspace where exported, or the portal sign-in-log view) for the CA-evaluation and troubleshooting labs.

• Helpers authenticate **interactively** with an MFA-capable account; **no credentials are stored in code**. Government-cloud tenants pass the matching endpoints — *verify endpoints for your cloud*.

2. Requirements Guide — Self-Deploy: Multi-Factor Authentication & Conditional Access

These are the **solution-specific** prerequisites for running the MFA & Conditional Access enablement kit. The **category baseline** — an active Microsoft Entra tenant, Entra licensing, least-privilege admin roles, break-glass accounts, and network access — is covered once in `global-requirements-entra-id-solutions.md` (**G1–G8**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. Entra ID licensing for Conditional Access (P1; P2 for risk labs)

• **Description: Microsoft Entra ID P1** for Conditional Access itself; **Entra ID P2** additionally if you want to *discuss/observe* the risk-based conditions (sign-in risk, user risk from Identity Protection) in the design lab.

• **Why required:** CA is a P1 feature; the report-only lab policies (Lab 4) need P1 to be created. Risk conditions need P2. *Verify current availability — bundles change and differ by cloud.*

• **How to verify:** M365 admin center -> **Billing -> Licenses**; or `EKMFACA-Discovery.ps1` reports the detected P1/P2 tier.

• **How to implement if missing:** Assign M365 E3/E5, EMS E3/E5, or standalone P1/P2. If the tenant has no P1, Labs 2–6 become a **read-only tour** of the CA and sign-in-log blades (no policy build) — note the gap; the MFA-methods lab still runs in full.

R2. A learner cohort and least-privilege lab roles

• **Description:** A named group of learners — `Grp-MfaCa-Cohort` — each holding a **read** role (**Global Reader** / **Security Reader**). The instructor holds **Conditional Access Administrator** (or **Security Administrator**) to *create* the report-only lab policies, and **Authentication Policy Administrator** to view/adjust the methods policy.

• **Why required:** Labs run at **least privilege** — learners read and follow along; only the instructor creates the report-only lab policies. This models the real least-privilege discipline the kit teaches.

• **How to verify:** Entra -> **Roles and administrators**; confirm the cohort holds read roles and the instructor holds `Conditional Access Administrator`. `EKMFACA-Discovery.ps1` lists role assignments.

• **How to implement if missing:** Create `Grp-MfaCa-Cohort`; assign read roles to learners; assign the instructor `Conditional Access Administrator` (prefer **PIM-eligible** where Entra ID P2 is available — *verify licensing*).

R3. Lab persona groups (the lab scope)

• **Description:** A small set of **cloud-only lab persona groups** the report-only policies target — `Grp-MfaCa-Persona-Internals-Pilot`, `Grp-MfaCa-Persona-Admins-Pilot`, `Grp-MfaCa-Persona-Guests-Pilot` — each containing only a few disposable test users (or the cohort members themselves for observation).

• **Why required:** The kit teaches **design by persona**; the report-only policies scope to these groups so the labs are legible, contained, and obviously safe (report-only + a tiny scope).

• **How to verify:** The groups exist with a handful of test members; `EKMFACA-Discovery.ps1` / `EKMFACA-Validate.ps1` list them.

• **How to implement if missing:** Create them (or run `EKMFACA-LabPolicy.ps1`, which creates the persona groups if absent before previewing the policies). Use cloud-only test users, never production staff.

R4. Break-glass accounts and an exclusion group

• **Description:** Two cloud-only emergency-access (break-glass) accounts (global requirement **G4**) and a dedicated exclusion group `Grp-MfaCa-Excl-BreakGlass` containing them, excluded from **every** lab policy.

• **Why required:** Even in report-only, the kit teaches the anti-lockout guarantee — break-glass excluded from every policy — so the habit is second nature before anyone builds an enforced policy (ENTZTCA).

• **How to verify:** Entra -> **Users**; two cloud-only `@onmicrosoft.com` admins exist and are members of `Grp-MfaCa-Excl-BreakGlass`. `EKMFACA-Discovery.ps1` flags whether break-glass-style accounts are present; `EKMFACA-Validate.ps1` checks every lab policy excludes the group.

• **How to implement if missing:** Create the accounts (**G4**) and the exclusion group; the lab policies exclude the group from the first save.

R5. MFA methods understood; sign-in logs available

• **Description:** Access to **Protection -> Authentication methods** (to see which methods are enabled and registration coverage) and to **Monitoring -> Sign-in logs** (for the CA-evaluation and troubleshooting labs), ideally exported to a **Log Analytics workspace** for the KQL labs.

• **Why required:** Lab 1 explores MFA methods and registration; Lab 6 reads sign-in failures and CA results — both need these blades. The KQL pack runs over exported sign-in logs.

• **How to verify:** The Authentication methods blade loads and shows enabled methods; Sign-in logs return recent rows. `EKMFACA-Discovery.ps1` reports the enabled methods and MFA-registration coverage.

• **How to implement if missing:** Grant read access to the blades; export sign-in logs to a workspace (**Diagnostic settings**) for the KQL labs, or use the portal sign-in-log view. *Sign-in-log export and retention depend on P1/P2 — verify.*

R6. The PowerShell + KQL helper toolchain

• **Description:** A workstation with the **Microsoft Graph PowerShell SDK**, signed in with an MFA-capable account holding at least read scopes (and, for the instructor, `Policy.ReadWrite.ConditionalAccess` for the report-only lab-policy builder); plus access to sign-in logs for the KQL labs.

• **Why required:** The five delivered helpers check readiness, build the report-only lab policies (gated), run the sign-in/CA KQL context, validate skills, and report impact + progress.

• **How to verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version; `Connect-MgGraph` succeeds; sign-in logs load.

• **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`; sign in. For Government cloud, connect with the matching environment/endpoints — *verify the correct endpoints for your cloud*.

Checklist

- Entra ID P1 confirmed (P2 to observe risk conditions) (**R1**)
- `Grp-MfaCa-Cohort` created; learners hold read roles; instructor holds `Conditional Access Administrator` (**R2**)
- Lab persona groups created with test members (**R3**)
- Break-glass accounts + `Grp-MfaCa-Excl-BreakGlass` created; excluded from every lab policy (**R4/G4**)
- Authentication methods + Sign-in logs reachable; export to workspace for KQL labs (**R5**)
- Microsoft Graph SDK installed and consented (**R6**)

Global Requirements — Entra ID Solutions (category-level)

These requirements apply to every solution in the Entra ID Solutions category. They are published once at the category level so individual listings stay small. Confirm these before deploying any identity solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *The identity plane is high risk. A mistake here breaks sign-in for everyone, so every change is staged and a break-glass account is kept ready before you touch anything.*

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID tenant. Why: It is the cloud identity backbone every other identity service builds on. Verify: Sign in to `https://entra.microsoft.com` as an administrator. Implement: Create a Microsoft 365 or Azure subscription, which provisions a tenant.

G2. Entra ID licensing (P1, P2 where noted)

Description: Microsoft Entra ID P1 (or P2 for risk-based and PIM features). Why: Hybrid features such as seamless sign-in, password writeback, and Conditional Access require P1+. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2 (often bundled in M365 E3/E5 or EMS). Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

G3. Administrative roles

Description: Hybrid Identity Administrator and Global Administrator for the operator. Why: Configuring directory synchronization and authentication methods needs these elevated roles. Verify: Entra -> `Roles and administrators`; confirm `Hybrid Identity Administrator` (and a Global Administrator for the one-time first-sync consent). Implement: Assign `Hybrid Identity Administrator`; reserve `Global Administrator` for tenant-wide and first-sync changes.

G4. Break-glass (emergency access) accounts

Description: At least two cloud-only emergency access accounts, excluded from Conditional Access and MFA-broad policies. Why: If sync, federation, or an auth-method change locks out admins, break-glass accounts get you back in. Verify: Entra -> `Users`; confirm two cloud-only accounts (`@onmicrosoft.com`) with strong stored credentials and documented use. Implement: Create two cloud-only Global

Administrator accounts; exclude them from policies; store credentials securely and monitor sign-ins.

G5. Healthy on-premises Active Directory

Description: A functional AD forest/domain with healthy replication and DNS. Why: Synchronization reads from AD; replication or DNS faults corrupt or stall the sync. Verify: Run `dcdiag` and `repadmin /replsummary` on a domain controller with no errors; run `HYBID-Discovery.ps1`. Implement: Resolve AD replication, DNS, and time-sync issues before synchronizing.

G6. An Entra Connect host server

Description: A domain-joined Windows Server (2016+), patched, for Microsoft Entra Connect Sync (or a lightweight host with the Cloud Sync agent). Why: Connect Sync runs as a service on this host and must reach both AD and Entra ID. Verify: A supported, domain-joined server with TLS 1.2, .NET, and PowerShell available; run `HYBID-Discovery.ps1`. Implement: Provision a dedicated server (not a domain controller for production), patch it, and enable TLS 1.2.

G7. UPN / domain hygiene

Description: User principal names use a public, verified, routable domain (not `.local`). Why: A non-routable UPN suffix produces sign-in problems and `onmicrosoft.com` fallbacks in the cloud. Verify: Entra -> `Identity -> Settings -> Domain names` shows the public domain Verified; AD UPN suffixes match it; run `HYBID-Discovery.ps1`. Implement: Add and verify the public domain in Entra; add the matching UPN suffix in AD and update user UPNs.

G8. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS (443) from the Connect host to Entra ID, plus required Microsoft 365 service URLs. Why: Synchronization, authentication-agent traffic, and tokens depend on these endpoints. Verify: From the Connect host, reach `https://login.microsoftonline.com` and the Microsoft 365 published URL/IP ranges. Implement: Allow the Microsoft 365 / Entra service URLs and IP ranges through the proxy/firewall.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Self-Deploy: Multi-Factor Authentication & Conditional Access

This is the exact, ordered guide — a sequence of **guided hands-on labs**. Every object you create is **named** with a consistent convention — `<Purpose> - <Scope> - <Stage>` for groups and `CALab0nn <Purpose> - <Persona> - ReportOnly` for policies — and the same name is reused in later labs, the knowledge-check, and the scripts. **Every lab Conditional Access policy is created in report-only and can never enforce**, break-glass is excluded from every policy from the first save, and the lab scope is a handful of cloud-only test users. No policy in this kit is ever enforced — enforcing a real baseline is the ENTZTCA / Conditional Access Integration solutions.

> ***Before you start:*** *confirm CA licensing (**R1**, P1), the cohort and least-privilege roles (**R2**), the lab persona groups (**R3**), break-glass and the exclusion group (**R4**, **G4**), the methods/sign-in-log access (**R5**), and the toolchain (**R6**). This guide **designs and tests** in report-only; it does not enforce.*

Object naming convention (used everywhere below and in the scripts). Create these once and reuse the exact names in testing, troubleshooting, and scripts:

Object	Name	What it is
Learner cohort group	`Grp-MfaCa-Cohort`	Entra security group of kit learners
Break-glass exclusion group	`Grp-MfaCa-Excl-BreakGlass`	Holds the two break-glass accounts; excluded from every policy
Admins persona (lab)	`Grp-MfaCa-Persona-Admins-Pilot`	Cloud-only test admins
Internals persona (lab)	`Grp-MfaCa-Persona-Internals-Pilot`	Cloud-only test staff
Guests persona (lab)	`Grp-MfaCa-Persona-Guests-Pilot`	Cloud-only test guests
Knowledge-check	`Check-MfaCa-Pilot`	The scored end-of-kit knowledge-check

Report-only lab policies (each created **report-only**, never enforced):

#	Policy	Persona	Control	License
CALab001	`CALab001 Require MFA - Internals - ReportOnly`	Internals	Require MFA	P1
CALab002	`CALab002 Require Phishing-Resistant MFA - Admins - ReportOnly`	Admins	Auth strength: phishing-resistant	P1
CALab003	`CALab003 Block Legacy Authentication - AllUsers - ReportOnly`	All (break-glass excluded)	Block legacy clients	P1

> *Every one is **report-only** and excludes `Grp-MfaCa-Excl-BreakGlass`. If the tenant has no P1, Labs 2–6 are a **read-only tour** instead of a policy build (**R1**). The runnable CA-evaluation and troubleshooting queries live in the delivered `EKMFACA-Labs.kql` pack.*

Phase 0 — Assess (read-only, changes nothing)

Step 0.1 — Confirm readiness

1. Sign in to the **Microsoft Entra admin center** -> `https://entra.microsoft.com`.

2. From PowerShell, take a read-only picture:

```
pwsh -File .\EKMFACA-Discovery.ps1
```

It reports, read-only: enabled MFA methods and registration coverage, existing Conditional Access policies and their states, which policies exclude break-glass (and flags any "all users, no exclusions" lock-out risk), detected legacy-auth sign-ins, and the licensing tier (P1/P2). **Nothing is modified.** Save the output — your before picture and a teaching prop.

Step 0.2 — Create the cohort

1. Entra -> **Groups -> New group** -> Security -> `Grp-MfaCa-Cohort`; add the learners.

2. Confirm learners hold read roles and the instructor holds `Conditional Access Administrator` (**R2**).

Phase 1 — Configure (persona groups + break-glass exclusion)

Step 1.1 — Create the lab persona and exclusion groups

1. Entra -> **Groups** -> create `Grp-MfaCa-Excl-BreakGlass` and add the two break-glass accounts (**R4/G4**).

1. Create the cloud-only persona groups `Grp-MfaCa-Persona-Admins-Pilot`, `Grp-MfaCa-Persona-Internals-Pilot`, `Grp-MfaCa-Persona-Guests-Pilot` and add a few disposable test users to each.

1. Or preview and create them with `EKMFACA-LabPolicy.ps1` (it creates the persona/exclusion groups if absent before previewing the report-only policies).

Phase 2 — Pilot (the guided labs, all report-only)

Run each lab with the cohort. Each lab states the **concept**, the **task**, and the **check**.

Lab 1 — MFA methods and registration

• **Concept.** MFA methods differ in strength: SMS/voice (weak, phishable) < authenticator with number matching < **phishing-resistant** (FIDO2/passkey, Windows Hello for Business, certificate-based). The **authentication-methods policy** controls what is enabled and drives registration.

• **Task.** Open **Protection -> Authentication methods -> Policies**; read which methods are enabled and for whom. Open **Authentication methods -> Activity / Registration** and read the registration coverage (or run `EKMFACA-Discovery.ps1`'s methods section). Discuss which methods your org should prefer. *Read-only — do not change the tenant methods policy in the lab.*

• **Check.** You can rank the MFA methods by strength, explain phishing-resistant, and find where methods and registration are configured.

Lab 2 — Anatomy of a Conditional Access policy

• **Concept.** A CA policy is if-then: **assignments/conditions** (users/groups + exclusions, target resources, device/location/client/risk) -> **grant controls** (require MFA / auth strength / compliant device / block) and/or **session controls** (sign-in frequency, persistent browser, app-enforced restrictions).

• **Task.** Open **Protection -> Conditional Access -> Policies**; open an existing policy (or start a New policy without saving) and walk each section: Users (include/exclude), Target resources, Conditions, Grant, Session, and the **Enable policy** toggle (On / Off / **Report-only**). *Do not save an enforced policy.*

• **Check.** You can name every section of a CA policy and explain what report-only does.

Lab 3 — Design patterns and personas

• **Concept.** Design CA by **persona**, not one mega-policy: **admins** (phishing-resistant MFA, tight session), **internals** (MFA + compliant/hybrid device), **guests** (MFA + app-enforced restrictions), **service accounts** (non-interactive — scoped out, governed separately). Personas map to groups; policies target groups.

• **Task.** Map your org's users onto the four personas on paper; match each to your lab persona group (`Grp-MfaCa-Persona-*-Pilot`). Sketch which control each persona needs. Discuss the common baseline patterns (MFA everywhere, phishing-resistant for admins, block legacy auth).

• **Check.** You can assign a user to the right persona and name the control that persona needs.

Lab 4 — Build the report-only policies (no lockouts)

• **Concept.** Every policy is created in **report-only** and excludes break-glass — it evaluates and logs but cannot block anyone.

• **Task.** Preview the report-only lab policies (changes nothing):

```
pwsh -File .\EKMFACA-LabPolicy.ps1
```

Review, then create them (still report-only):

```
pwsh -File .\EKMFACA-LabPolicy.ps1 -WhatIf:$false
```

This creates `CALab001` (Require MFA - Internals), `CALab002` (Phishing-Resistant MFA - Admins), and `CALab003` (Block Legacy - All Users) — all **report-only**, all scoped to the persona groups, all excluding `Grp-MfaCa-Excl-BreakGlass`. *You can also build each by hand in the portal: New policy -> name it exactly `CALab00n ... - ReportOnly`, set the persona include, the break-glass exclude, the grant control, and **Enable policy = Report-only**.* If no P1, do this as a read-only tour (**R1**).

• **Check.** The three lab policies exist, are **report-only**, scoped to the right persona, and each excludes break-glass.

Lab 5 — Test with What-If (simulate before enforce)

• **Concept. What-If** simulates a sign-in and shows exactly which policies would apply and which controls would fire — with no real sign-in and no risk.

• **Task.** Open **Conditional Access -> What If**. Run it for: a test **internal** user (expect `CALab001` to apply), a test **admin** (expect `CALab002`), and a **break-glass** account (expect **no** policy to apply — the exclusion works). Adjust scope/exclusions until What-If matches the design.

• **Check.** What-If shows each persona hitting its intended policy and break-glass hitting **none**.

Lab 6 — Troubleshoot sign-in failures and CA evaluation

• **Concept.** Every sign-in logs **why** each CA policy applied or did not (success / failure /

reportOnlySuccess / reportOnlyFailure / notApplied) and which MFA method satisfied the requirement. Most "MFA is broken" tickets are a policy doing exactly what it was told.

• **Task.** Open **Monitoring -> Sign-in logs**; open a sign-in and read its **Conditional Access**

tab (which policies applied and their result, including the report-only labs), **Authentication Details** (which method satisfied MFA), and the failure reason for any failed sign-in. Run the `EKMFACA-Labs.kql` queries (CA-evaluation and failure-reason sections) to summarize would-apply results and top failure reasons.

• **Check.** Given a sign-in, you can state which policy applied, why, and what the user must do — and

distinguish a real failure from a report-only "would have."

Phase 3 — Validate (capstone + knowledge-check)

1. **Capstone.** Each learner, unaided: designs a report-only policy for a chosen persona (or explains

one of `CALab001–003`), proves it with **What-If** for that persona **and** for break-glass (must apply to the persona, must **not** apply to break-glass), then reads a sign-in log to explain the result. *Everything stays report-only.*

1. **Knowledge-check.** Score `Check-MfaCa-Pilot`:

```
pwsh -File .\EKMFACA-Validate.ps1 -BreakGlassUpn bg1@contoso.onmicrosoft.com,bg2@contoso.onmicrosoft.co
```

It walks the pass/fail gates in `05-testing.md` — lab policies exist and are **report-only**, break-glass is excluded from every one, personas are scoped correctly — and prints the knowledge-check answer key for the instructor to score.

1. **Impact + cohort report.** Summarize what the report-only policies *would* have done and cohort

progress:

```
pwsh -File .\EKMFACA-Report.ps1 -OutputCsv .\ekmfaca-cohort.csv
```

Phase 4 — Expand (take the capability to production)

1. **Move to the enforced baseline solutions.** With the team able to design, test, and troubleshoot

MFA/CA, the natural next step is the **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** — design and roll out a real, **enforced** baseline (report-only -> pilot -> enforce, wave by wave). This kit built the skill; those products build the production system.

1. **Deploy phishing-resistant methods.** Feed the MFA-methods understanding into the
**Passwordless &

Passkey (ENTPASSKEY)** rollout so `CALab002`-style phishing-resistant requirements have a method to satisfy them.

1. **Adopt the runbook.** Move the take-home runbook into standing operations documentation so the

MFA/CA design-test-troubleshoot method is reused, not re-explained.

1. **Tear down or keep the lab.** Remove the report-only lab policies and persona groups if the kit was one-off, or keep them for re-runs.

Rollback / safe-off

- **Concept/observation labs (1, 2, 3, 5, 6):** read-only — nothing to roll back.
- **Report-only lab policies (Lab 4):** they are **report-only** and cannot enforce; set each to `Off` or delete `CALab001–003` (`EKMFACA-LabPolicy.ps1 -Remove -WhatIf:$false` after previewing). Break-glass is excluded throughout, so there is no lock-out risk at any point.
- **Persona/exclusion groups:** delete the `Grp-MfaCa-*` groups if the kit was one-off; keep them if operationalizing into the ENTZTCA baseline.

5. Testing & Validation — Self-Deploy: Multi-Factor Authentication & Conditional Access

Validate the kit against explicit **pass/fail gates** — both that the **lab is set up safely** and that the **skills** landed. Run the scripts and walk the manual checks. The automated gates are produced by `EKMFACA-Validate.ps1`; readiness comes from `EKMFACA-Discovery.ps1`, the CA-evaluation/troubleshooting labs from `EKMFACA-Labs.kql`, and impact + cohort evidence from `EKMFACA-Report.ps1`.

Each gate lists **What good looks like / How to test / If it fails**.

Gate 1 — CA is available and the MFA/CA picture is captured

- **What good looks like:** The Entra admin center loads; `EKMFACA-Discovery.ps1` returns enabled MFA methods, registration coverage, existing CA policies and states, and the P1/P2 tier.
- **How to test:** Run `EKMFACA-Discovery.ps1`; confirm it prints the methods, CA inventory, and licensing without error.
- **If it fails:** Confirm P1 (**R1**), the Graph SDK is installed and consented (**R6**), and the account holds a read role (**R2**). If no P1, Labs 2–6 are a read-only tour; the MFA-methods lab still runs.

Gate 2 — Cohort, persona groups, and break-glass exclusion exist

- **What good looks like:** `Grp-MfaCa-Cohort`, the three `Grp-MfaCa-Persona-*-Pilot` groups, and `Grp-MfaCa-Excl-BreakGlass` (containing the two break-glass accounts) all exist.
- **How to test:** Entra -> **Groups**, or run `EKMFACA-Validate.ps1`, which lists each group and the

break-glass membership.

• **If it fails:** Create the missing groups (Step 0.2 / 1.1) or run `EKMFACA-LabPolicy.ps1`, which creates the persona/exclusion groups if absent.

Gate 3 — The report-only lab policies exist and are report-only

• **What good looks like:** `CALab001`, `CALab002`, and `CALab003` exist, each with **Enable policy = Report-only** (`enabledForReportingButNotEnforced`), scoped to the correct persona.

• **How to test:** Run `EKMFACA-Validate.ps1`; confirm each lab policy is present, report-only, and correctly scoped. *If no P1, mark N/A and confirm the read-only CA tour instead.*

• **If it fails:** Set any drifted policy back to **Report-only** immediately (it must never enforce in this kit); re-scope to the correct persona group (Lab 4).

Gate 4 — Break-glass is excluded from every lab policy

• **What good looks like:** `Grp-MfaCa-Excl-BreakGlass` (or the break-glass users) is excluded from `CALab001`, `CALab002`, and `CALab003`.

• **How to test:** `EKMFACA-Validate.ps1 -BreakGlassUpn ...` reports each policy's exclusions and flags any missing the break-glass exclusion in red. Or check **Assignments -> Users -> Exclude** per policy.

• **If it fails:** Add `Grp-MfaCa-Excl-BreakGlass` to the policy's exclusions (**R4/G4**). This is the anti-lockout habit the kit exists to teach.

Gate 5 — What-If matches the persona design

• **What good looks like:** In **Conditional Access -> What If**, a test internal user hits `CALab001`, a test admin hits `CALab002`, and a break-glass account hits **no** policy.

• **How to test:** Run What-If for one representative user per persona and for a break-glass account; confirm the applied policies match the design.

• **If it fails:** Fix the include/exclude scope until What-If matches (Lab 5); a break-glass account applying to any policy means the exclusion is missing (Gate 4).

Gate 6 — Learners can read a sign-in and explain the CA decision

• **What good looks like:** Given a sign-in in the logs, a learner can state which CA policies applied, their result (including report-only "would have"), which MFA method satisfied the requirement, and what a failed user must do.

• **How to test:** Observe the capstone; have the learner open a sign-in (or run an `EKMFACA-Labs.kql` query) and narrate the CA evaluation and any failure reason.

• **If it fails:** Re-run Lab 6 (troubleshooting); reinforce the difference between a real failure and a report-only result.

Gate 7 — The knowledge-check passes

• **What good looks like:** Each learner passes `Check-MfaCa-Pilot` (MFA method strength, CA anatomy, personas, report-only vs enforce, What-If, break-glass, reading a sign-in result).

• **How to test:** Run `EKMFACA-Validate.ps1`; it prints the knowledge-check answer key mapped to the lab that teaches each item, for the instructor to score.

• **If it fails:** Revisit the mapped lab for any question a learner missed; the check tests **concepts**, not tenant-specific values.

Gate 8 — Report-only-only posture held; roles least-privilege

• **What good looks like:** No lab policy was ever enforced; learners held read roles and only the instructor created policies; break-glass was excluded from every policy throughout.

• **How to test:** `EKMFACA-Validate.ps1` confirms every lab policy is still report-only; review **Roles and administrators** (R2).

• **If it fails:** Set any enforced policy back to report-only immediately; tighten role assignments; reaffirm the report-only posture.

Validation summary

Gate	Check	Pass condition
1	CA available + picture captured	`EKMFACA-Discovery.ps1` returns methods + CA + licensing
2	Cohort + personas + break-glass group	All groups present; break-glass in the exclusion group
3	Lab policies exist + report-only	`CALab001–003` present, all report-only, correctly scoped
4	Break-glass excluded everywhere	Every lab policy excludes the break-glass group
5	What-If matches design	Each persona hits its policy; break-glass hits none
6	Read a sign-in + explain CA	Capstone: CA decision + method + failure reason narrated

Gate	Check	Pass condition
7	Knowledge-check passes	`Check-MfaCa-Pilot` scored to threshold
8	Report-only-only + least-privilege	No policy enforced; roles scoped

All gates pass ⇒ the cohort can design, test, and troubleshoot MFA/CA safely; proceed to **Expand** (the enforced-baseline solutions). Any fail ⇒ reinforce the mapped lab and re-check.

6. Troubleshooting — Self-Deploy: Multi-Factor Authentication & Conditional Access

The common failure modes, each as **symptom** → **cause** → **fix**. Most kit issues are a missing MFA registration, a policy not in report-only, a missing break-glass exclusion, or misreading a report-only result as a real failure — not a broken tenant. Use `EKMFACA-Discovery.ps1` for the read-only picture and `EKMFACA-Validate.ps1` for the gate checks.

1. `Connect-MgGraph` fails or a script returns 403

- **Symptom:** A helper cannot connect, or a call is forbidden.
- **Cause:** The Graph SDK is not installed, scopes were not consented, the wrong cloud endpoint was used, or the account lacks the role.
- **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`; re-run and consent the scopes in the script header (read-only for discovery/validate/report; `Policy.ReadWrite.ConditionalAccess` for the lab-policy builder); for Government cloud pass the matching `-Environment`; confirm the account holds the needed role (**R2/R6**). *Verify the correct endpoints for your cloud.*

2. The report-only lab policy can't be created (CA blade missing)

- **Symptom:** `EKMFACA-LabPolicy.ps1` fails, or **Conditional Access** is unavailable in the portal.
- **Cause:** The tenant has no **Entra ID P1** (Conditional Access is a P1 feature), or the account lacks `Conditional Access Administrator`.
- **Fix:** Confirm P1 (**R1**) and the instructor role (**R2**). If there is no P1, run Labs 2–6 as a **read-only tour** of the CA and sign-in-log blades and note the gap; the MFA-methods lab (Lab 1) still runs in full. *Verify licensing.*

3. A lab policy is not in report-only

- **Symptom:** `EKMFACA-Validate.ps1` flags `CALab00n` as On or Off instead of report-only.
- **Cause:** The state was changed.

• **Fix:** Set **Enable policy = Report-only** immediately. In this kit **no lab policy is ever enforced** — report-only is mandatory. If it was briefly On, confirm no user was unexpectedly blocked (sign-in logs) and reset to report-only.

4. Break-glass is not excluded from a lab policy

• **Symptom:** `EKMFACA-Validate.ps1` flags a lab policy as missing the break-glass exclusion, or What-If shows a policy applying to a break-glass account.

• **Cause:** The exclusion group was not added to that policy.

• **Fix:** Add `Grp-MfaCa-Excl-BreakGlass` under **Assignments -> Users -> Exclude** on every lab policy (**R4/G4**). Exclude the **group**, not the individual users — a group exclusion is durable. This is the core anti-lockout habit the kit teaches.

5. What-If shows a policy applying to the wrong persona (or not at all)

• **Symptom:** In Lab 5, What-If doesn't match the design — an internal user doesn't hit `CALab001`, or an admin also hits an internals policy.

• **Cause:** The user is in the wrong persona group, the policy's include/exclude scope is off, or a condition (device/location/client) filters it out.

• **Fix:** Confirm the test user's persona-group membership; re-check the policy's **Users** include and **exclude** and its conditions; re-run What-If until it matches (Lab 5). Remember dynamic-group membership can lag.

6. A "failure" in the sign-in log is actually a report-only result

• **Symptom:** A learner reports the lab policy "blocked" a user.

• **Cause:** The sign-in shows `reportOnlyFailure` — the policy **would have** required/blocked, but because it is report-only it did **not** enforce.

• **Fix:** Teach the distinction (Lab 6): `success`/`failure` are enforced results; `reportOnlySuccess` / `reportOnlyFailure` / `reportOnlyNotApplied` are the report-only "would have." A report-only result is never an outage. Read the CA tab's per-policy result to confirm.

7. MFA prompts constantly / a phishing-resistant policy rejects an admin

• **Symptom:** A test admin under `CALab002` is asked for a method they don't have (in a real enforced version), or MFA prompts more than expected.

• **Cause:** The admin has not **registered** a phishing-resistant method (FIDO2/passkey), or sign-in frequency/session settings force re-prompts.

• **Fix:** In this kit the policy is report-only so it cannot actually block — but the lesson is real:

drive **registration** of the required method first (Lab 1; the **Passwordless & Passkey** solution deploys it at scale) before any enforced policy requires it. *Verify passkey/method availability.*

8. Sign-in logs / the KQL lab return no rows

- **Symptom:** An `EKMFACA-Labs.kql` query returns 0 rows, or Sign-in logs look empty.
- **Cause:** The time window is too tight, the test users haven't signed in, or sign-in logs aren't exported to a Log Analytics workspace (the KQL variant).
- **Fix:** Widen the window (`ago(7d)`); have the persona test users sign in once to generate report-only results; use the **Entra -> Monitoring -> Sign-in logs** portal view if no workspace export exists. *Sign-in-log export/retention depend on P1/P2 — verify.*

9. Legacy-authentication confusion

- **Symptom:** `CALab003` (block legacy) shows results for apps you didn't expect, or a learner worries it will break mail.
- **Cause:** Legacy/basic-auth clients (older mail clients, `Other clients`) surface in the logs; `CALab003` is **report-only** so it only *observes* them here.
- **Fix:** Use Lab 6 / `EKMFACA-Discovery.ps1` to identify legacy-auth users/apps. In this kit you only observe — actually blocking legacy auth in production (migrate first, then enforce in waves) is the **ENTZTCA** solution, not this one.

10. Dynamic persona-group membership didn't update

- **Symptom:** A test user added to a persona doesn't appear to be scoped by the policy yet.
- **Cause:** If the persona group is dynamic, membership is evaluated **asynchronously** and can lag; if assigned, the user may not have been added.
- **Fix:** For assigned groups, confirm the membership; for dynamic groups, wait and refresh and confirm the rule matches the user's attribute exactly. Re-run What-If (Lab 5) once membership settles.

7. Best Practices — Self-Deploy: Multi-Factor Authentication & Conditional Access

Keep-it-clean guidance for turning an MFA/CA kit into a durable design-test-troubleshoot skill — and where to go next once the capability is in place.

Report-only before enforce — always

- **Learn in report-only; enforce nowhere in this kit.** Every lab policy is created **report-only**;

it evaluates and logs but cannot block. Carry that discipline into every real policy you build later: report-only, validate with a few days of data and What-If, then enforce in waves.

- **What-If every policy against every persona.** Before a policy is ever enforced (in the production solutions), simulate it for each persona **and** for a break-glass account. A break-glass account applying to any policy means the exclusion is missing.

Protect break-glass, every policy, no exceptions

- **Two cloud-only emergency accounts, excluded from every CA policy.** Keep them in `Grp-MfaCa-Excl-BreakGlass` and exclude that group from every policy — `EKMFACA-Validate.ps1` flags any policy that forgets. Teach the habit in report-only so it is automatic before any enforced policy exists.
- **Exclude the group, not the users.** A group exclusion is durable; a per-user exclusion is fragile and easy to lose when accounts change.

Choose MFA methods by strength

- **Prefer phishing-resistant.** SMS/voice are phishable; push+number-matching is good; FIDO2/passkeys, Windows Hello for Business, and certificate-based auth are **phishing-resistant** — require them for admins via an **authentication strength** grant control. *Passkey/CBA support may be preview or licensing-gated — verify.*
- **Drive registration before you require.** A policy that requires a method users have not registered locks them out. Confirm registration coverage (Lab 1 / `EKMFACA-Discovery.ps1`) first — the **Passwordless & Passkey** solution deploys the method at scale.

Design by persona, not by mega-policy

- **Admins, internals, guests, service accounts.** A numbered, persona-based set is legible, auditable, and rolled out group by group; a single "all users, all apps" policy is a lock-out waiting to happen.
- **Target groups, not individuals.** Scope policies to persona security groups so the design is legible and scalable.
- **Handle service accounts deliberately.** Non-interactive accounts break under interactive MFA — scope them out of interactive policies and govern them separately (workload-identity CA where licensed), never a blanket silent exclusion.

Troubleshoot from the sign-in log, not from a guess

- **Read the CA tab.** Every sign-in records which policies applied and why (success / failure / reportOnly* / notApplied) and which method satisfied MFA. Most "MFA is broken" tickets are a policy doing exactly what it was told — the log tells you which one.

• **Distinguish report-only from real.** A `reportOnlyFailure` means the policy *would have* blocked —

it did not. Do not chase a report-only result as an outage.

Verify, don't assume

• **Verify licensing before you promise a feature.** CA needs **P1**; risk-based conditions and PIM

need **P2**; some MFA methods may be preview. Check **Billing -> Licenses** (or `EKMFACA-Discovery.ps1`) — *bundles change and differ by cloud (Commercial vs GCC / GCC High)*.

• **Verify portal navigation and preview status** against your tenant — Microsoft renames blades and

ships features in preview.

Run it as a program, not a one-off class

• **Re-run for new hires.** The take-home runbook and knowledge-check make onboarding a new security

admin repeatable; the persona groups and scripts are reusable.

• **Keep the runbook current** as your CA baseline evolves in the production solutions.

Where to go next

With the team able to design, test, and troubleshoot MFA/CA, the natural next steps in this catalog are:

• **Zero Trust Conditional Access Baseline (ENTZTCA)** and **Conditional Access Integration** — design

and roll out a real, **enforced** baseline (this kit built the skill in report-only; those build the production system).

• **Passwordless & Passkey (ENTPASSKEY)** — deploy the phishing-resistant methods that a

phishing-resistant auth-strength policy requires.

• **Privileged Identity Management (ENTPIM)** — pair CA with just-in-time, approval-gated admin

elevation.

• **Identity Protection (P2)** — add risk-based sign-in and user-risk conditions once the team is

comfortable with the fundamentals of CA.

• **Self-Deploy — Microsoft Entra Identity Fundamentals (EKIDFUND)** — the prerequisite grounding

for anyone on the team still shaky on tenants, users, groups, and roles.

8. Deliverables Summary — Self-Deploy: Multi-Factor Authentication & Conditional Access

Everything included in this kit, and what each piece is for.

Documentation

Document	Purpose
Executive Summary (`executive-summary.md`)	One-page framing: the "too timid or too reckless with MFA/CA" problem, the enablement solution (concepts + report-only labs + take-home runbook + knowledge-check), and the boundary (skills product; **distinct from** the ENTZTCA / Conditional Access Integration *implementation* solutions; practices report-only, never enforces).
1. Solution Overview (`01-solution-overview.md`)	What it is, how MFA methods and the Conditional Access model work (signals, conditions, grant/session controls, personas, report-only + What-If, break-glass), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used.
2. Requirements Guide (`02-requirements.md`)	Solution-specific prerequisites (CA licensing P1/P2, cohort + least-privilege roles, lab persona groups, break-glass + exclusion group, methods/sign-in-log access, toolchain) — layered on the shared `global-requirements-entra-id-solutions.md` (G1–G8).
Global Requirements (`global-requirements-entra-id-solutions.md`)	Category-wide prerequisites, published once (tenant, licensing, admin roles, break-glass, network access).
3. Implementation Guide (`03-implementation.md`)	The guided labs: assess readiness, create the cohort/persona/exclusion groups, run Labs 1–6 (MFA methods & registration → CA anatomy → design patterns & personas → build **report-only** policies → What-If testing → sign-in/CA-evaluation troubleshooting), then capstone + knowledge-check, then move to the enforced-baseline solutions — every object named, every policy report-only, with rollback.
5. Testing & Validation (`05-testing.md`)	Eight pass/fail gates covering safe lab setup and skills landed (lab policies report-only, break-glass excluded everywhere, What-If matches design, read a sign-in and explain CA, knowledge-check passes), referencing the scripts by name.
6. Troubleshooting (`06-troubleshooting.md`)	The common failure modes (MFA registration gaps, policy not report-only, break-glass not excluded, What-If surprises, report-only vs real failures, legacy-auth confusion, connection/permission errors) as symptom → cause → fix.

Document	Purpose
7. Best Practices (`07-best-practices.md`)	Report-only-before-enforce, protect-break-glass, choose-methods-by-strength, design-by-persona, troubleshoot-from-the-log, verify-don't-assume, run-it-as-a-program — plus where to go next in the catalog.

Lab helpers (5)

All helpers authenticate **interactively** (no credentials in code) and use the **Microsoft Graph PowerShell SDK**; the KQL pack runs over Entra sign-in logs. The discovery, validation, and report helpers are **read-only**; the lab-policy builder is **preview (`-WhatIf`) by default**, gated, and creates **report-only policies only** — it can never enforce. Government-cloud tenants pass the matching environment — *verify endpoints for your cloud*.

File	Type	What it does
`EKMFACA-Discovery.ps1`	Read-only	Inventories the MFA/CA picture: enabled MFA methods and registration coverage, existing Conditional Access policies and states, which policies exclude break-glass (flags "all users, no exclusions" lock-out risk), detected legacy-auth sign-ins, and the licensing tier (P1/P2). Changes nothing.
`EKMFACA-LabPolicy.ps1`	Gated write (preview by default)	Creates the persona/exclusion groups if absent, then the report-only lab policies `CALab001` (Require MFA - Internals), `CALab002` (Phishing-Resistant MFA - Admins), `CALab003` (Block Legacy - All Users) — every one **report-only** with break-glass excluded. Defaults to `-WhatIf`; `-Remove` tears the lab down. Creates report-only ONLY; never enforces.

File	Type	What it does
`EKMFACA-Labs.kql`	KQL lab pack	The runnable sign-in-log query pack for the CA-evaluation and troubleshooting labs — CA policy "would-apply" results (incl. report-only), top sign-in failure reasons, which MFA method satisfied the requirement, and legacy-auth observation. Read-only; the take-home query starter.
`EKMFACA-Validate.ps1`	Read-only	Runs the `05-testing.md` gates as PASS/FAIL/MANUAL lines: lab policies present and **report-only**, break-glass excluded from every one, personas scoped correctly — and prints the knowledge-check `Check-MfaCa-Pilot` answer key for scoring.
`EKMFACA-Report.ps1`	Read-only	Reports what the report-only policies *would* have done (would-grant / would-require / would-block per policy, per persona) and cohort progress + MFA-registration coverage; exports to CSV.

Named objects (verbatim — used across docs, scripts, and tests)

- **Learner cohort group:** `Grp-MfaCa-Cohort`
- **Break-glass exclusion group:** `Grp-MfaCa-Excl-BreakGlass`
- **Lab persona groups:** `Grp-MfaCa-Persona-Admins-Pilot`, `Grp-MfaCa-Persona-Internals-Pilot`, `Grp-MfaCa-Persona-Guests-Pilot`
- **Report-only lab policies:** `CALab001 Require MFA - Internals - ReportOnly`, `CALab002 Require Phishing-Resistant MFA - Admins - ReportOnly`, `CALab003 Block Legacy Authentication - AllUsers - ReportOnly`
- **Knowledge-check:** `Check-MfaCa-Pilot`

What this kit deliberately does NOT do

- It does **not** design or roll out a production, enforced Conditional Access baseline — that is **ENTZTCA** and **Conditional Access Integration**. Every policy here is **report-only** and cannot enforce.
- It does **not** deploy phishing-resistant methods at scale — that is **Passwordless & Passkey

(ENTPASSKEY)**. It teaches the methods and cross-references.

- It does **not** build risk-based (P2), PIM, governance, or hybrid — those are **Identity

Protection**,** ENTPIM**,** ENTIGA**, and** Hybrid Identity**. It cross-references, not duplicates.

- It does **not** decide business/security policy — which MFA methods are mandated, which users are

"privileged," and every production enforcement decision are the customer's determinations; the kit provides the skill and the labs.

- It does **not** invent licensing or preview facts — where availability may differ by tenant/cloud

(P1/P2, passkeys, CBA, preview features) it says **verify in your tenant**.

How it is delivered

This kit is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the KQL lab pack and all five lab helpers, in a single ZIP (`EKMFACA-Complete-Solution.zip`) — yours to keep and re-run for every new cohort.

Outcome

A team that can **design, test, and troubleshoot** MFA and Conditional Access on Microsoft Entra safely — the right methods, persona-based policies, every policy proven in report-only and What-If before it can enforce, break-glass protected, and sign-in logs read with confidence — ready to move into the production enforced-baseline solutions.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EKMFACA-Discovery.ps1

Purpose: A read-only "before picture" for the MFA & Conditional Access enablement kit: enabled MFA methods (from the authentication-methods policy) and tenant-wide MFA-registration coverage, every existing Conditional Access policy and its state (enabled / disabled / report-only), which policies target "all users" and whether they exclude any accounts (a lock-out-risk flag), detected legacy-authentication sign-ins, and the licensing tier (Entra ID P1 / P2). This is a teaching prop and a readiness check — it changes NOTHING. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once pwsh -File .\EKMFACA-Discovery.ps1 pwsh -File .\EKMFACA-Discovery.ps1 -CsvPath .\ekmfaca-discovery.csv -LegacyLookbackDays 7 Consents to read-only scopes (Policy.Read.All, Directory.Read.All, AuditLog.Read.All, Reports.Read.All, Policy.Read.ConditionalAccess). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary of methods, registration coverage, CA policies + states + exclusions, legacy-auth sign-ins, and licensing. Nothing is modified. Optional CSV export. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([int]$LegacyLookbackDays = 7, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once pwsh -File .\EKMFACA-Discovery.ps1 pwsh -File .\EKMFACA-Discovery.ps1 -CsvPath .\ekmfaca-discovery.csv -LegacyLookbackDays 7 Consents to read-only scopes (Policy.Read.All, Directory.Read.All, AuditLog.Read.All, Reports.Read.All, Policy.Read.ConditionalAccess). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary of methods, registration coverage, CA policies + states + exclusions, legacy-auth sign-ins, and licensing. Nothing is modified. Optional CSV export. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([int]$LegacyLookbackDays = 7, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKMFACA-LabPolicy.ps1

Purpose: Creates the lab persona/exclusion groups if absent, then the three report-only Conditional Access lab policies used by the kit: CALab001 Require MFA - Internals - ReportOnly CALab002 Require Phishing-Resistant MFA - Admins - ReportOnly CALab003 Block Legacy Authentication - AllUsers - ReportOnly EVERY policy is created in REPORT-ONLY (enabledForReportingButNotEnforced) and excludes the break-glass group Grp-MfaCa-Excl-BreakGlass — so nothing can ever be blocked. This script can NEVER create an enforced policy. Enforcing a real baseline is the ENTZTCA / Conditional Access Integration solutions, not this kit. Defaults to -WhatIf (preview); a plain run changes nothing. -Remove tears the lab policies down (preview by default too). HOW TO RUN Preview (default, no changes): pwsh -File .\EKMFACA-LabPolicy.ps1 Create the report-only lab (after review): pwsh -File .\EKMFACA-LabPolicy.ps1 -WhatIf:$false Tear the lab policies down (preview, then apply): pwsh -File .\EKMFACA-LabPolicy.ps1 -Remove pwsh -File .\EKMFACA-LabPolicy.ps1 -Remove -WhatIf:$false Consents to Policy.ReadWrite.ConditionalAccess, Group.ReadWrite.All, Policy.Read.All, Directory.Read.All. Government cloud: -Environment USGov. EXPECTED RESULTS In preview: the groups/policies that WOULD be created (all report-only). When applied: the three report-only policies exist, persona-scoped, break-glass excluded. Fully reversible. .NOTES White-label — Entra ID Solutions. Write

action — preview (-WhatIf) by default. Creates REPORT-ONLY policies ONLY; never enforces. No credentials in code. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$Remove, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: Preview (default, no changes): pwsh -File .\EKMFACA-LabPolicy.ps1 Create the report-only lab (after review): pwsh -File .\EKMFACA-LabPolicy.ps1 -WhatIf:$false Tear the lab policies down (preview, then apply): pwsh -File .\EKMFACA-LabPolicy.ps1 -Remove pwsh -File .\EKMFACA-LabPolicy.ps1 -Remove -WhatIf:$false Consents to Policy.ReadWrite.ConditionalAccess, Group.ReadWrite.All, Policy.Read.All, Directory.Read.All. Government cloud: -Environment USGov. EXPECTED RESULTS In preview: the groups/policies that WOULD be created (all report-only). When applied: the three report-only policies exist, persona-scoped, break-glass excluded. Fully reversible. .NOTES White-label — Entra ID Solutions. Write action — preview (-WhatIf) by default. Creates REPORT-ONLY policies ONLY; never enforces. No credentials in code. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$Remove, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKMFACA-Labs.kql

Purpose: // The runnable KQL for the CA-evaluation and troubleshooting labs (Lab 6). One labeled // section per task. This is a take-home starter — keep it and grow it into your team's // identity troubleshooting library. // HOW TO RUN // These run over Entra sign-in logs. Two options: // - Log Analytics: if you export Entra sign-in logs to a workspace (Diagnostic settings // -> SigninLogs), paste a query into Sentinel/Log Analytics -> Logs. // - Portal filters: if you do NOT export to a workspace, use the equivalent filter in // Entra -> Monitoring -> Sign-in logs (each section notes the filter). // The read-only PowerShell helpers (EKMFACA-Discovery/Report) cover the same signals via Graph. // EXPECTED RESULTS // Small, explainable result sets you can read in class. Where a column may differ in your // tenant/cloud it is flagged // VERIFY. // NOTES // White-label. READ-ONLY — every query only reads logs; nothing changes the tenant. Report-only // CA results appear as reportOnlySuccess / reportOnlyFailure / reportOnlyNotApplied — these are // "would have", NOT enforced. Sign-in-log export needs Entra ID P1/P2 + a Diagnostic setting — // VERIFY in your tenant. Column names evolve — VERIFY current schema (Logs -> schema). // ==

How to run: // These run over Entra sign-in logs. Two options: // - Log Analytics: if you export Entra sign-in logs to a workspace (Diagnostic settings // -> SigninLogs), paste a query into Sentinel/Log Analytics -> Logs. // - Portal filters: if you do NOT export to a workspace, use the equivalent filter in // Entra -> Monitoring -> Sign-in logs (each section notes the filter). // The read-only PowerShell helpers (EKMFACA-Discovery/Report) cover the same signals via Graph. // EXPECTED RESULTS // Small, explainable result sets you can read in class. Where a column may differ in your // tenant/cloud it is flagged // VERIFY. // NOTES // White-label. READ-ONLY — every query only reads logs; nothing changes the tenant. Report-only // CA results appear as reportOnlySuccess / reportOnlyFailure / reportOnlyNotApplied — these are // "would have", NOT enforced. Sign-in-log export needs Entra ID P1/P2 + a Diagnostic setting — // VERIFY in your tenant. Column names evolve — VERIFY current schema (Logs -> schema). // ==

EKMFACA-Report.ps1

Purpose: The evidence report the instructor runs at the end of the kit. From recent sign-in logs it summarizes what the REPORT-ONLY lab policies (CALab001-003) WOULD have done — per policy: how many sign-ins they would have granted, would have required a control for, or would have blocked — so the cohort sees impact without anything being enforced. It also reports the cohort roster, which lab policies/groups exist, and tenant-wide MFA-registration coverage. Read-only. HOW TO RUN pwsh -File .\EKMFACA-Report.ps1 pwsh -File .\EKMFACA-Report.ps1 -LookbackDays 7 -OutputCsv .\ekmfaca-cohort.csv Consents to read-only scopes (AuditLog.Read.All, Policy.Read.All, Directory.Read.All, Reports.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary + optional CSV. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([int]$LookbackDays = 7, [string]$OutputCsv, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: pwsh -File .\EKMFACA-Report.ps1 pwsh -File .\EKMFACA-Report.ps1 -LookbackDays 7 -OutputCsv .\ekmfaca-cohort.csv Consents to read-only scopes (AuditLog.Read.All, Policy.Read.All, Directory.Read.All, Reports.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A console summary + optional CSV. Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([int]$LookbackDays = 7, [string]$OutputCsv, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

EKMFACA-Validate.ps1

Purpose: Turns 05-testing.md into a checklist run: confirms the cohort/persona/exclusion groups exist, that the report-only lab policies CALab001/002/003 exist and are REPORT-ONLY, that break-glass is excluded from every one, and the licensing tier — then prints the knowledge-check Check-MfaCa-Pilot answer key for the instructor to score. Read-only. HOW TO RUN pwsh -File .\EKMFACA-Validate.ps1 pwsh -File .\EKMFACA-Validate.ps1 -BreakGlassUpn bg1@contoso.onmicrosoft.com,bg2@contoso.onmicrosoft.com Consents to read-only scopes (Directory.Read.All, Policy.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A list of PASS / FAIL / MANUAL gate lines and the knowledge-check answer key. Any FAIL = reinforce the mapped lab and re-check. Any lab policy NOT report-only is a hard FAIL (this kit never enforces). Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string[]]$BreakGlassUpn, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: pwsh -File .\EKMFACA-Validate.ps1 pwsh -File .\EKMFACA-Validate.ps1 -BreakGlassUpn bg1@contoso.onmicrosoft.com,bg2@contoso.onmicrosoft.com Consents to read-only scopes (Directory.Read.All, Policy.Read.All). Government cloud: -Environment USGov. EXPECTED RESULTS A list of PASS / FAIL / MANUAL gate lines and the knowledge-check answer key. Any FAIL = reinforce the mapped lab and re-check. Any lab policy NOT report-only is a hard FAIL (this kit never enforces). Nothing is modified. .NOTES White-label — Entra ID Solutions. Read-only. > [CmdletBinding()] param([string[]]$BreakGlassUpn, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')